

Versi 2.0

Januari 2026



PRAKTIKUM SISTEM OPERASI

MODUL 1 - PENDAHULUAN TENTANG

SISTEM OPERASI DAN STRUKTUR SISTEM KOMPUTER

Pengarang:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

Sistem operasi, Struktur Sistem Komputer, Arsitektur sistem

LATAR BELAKANG TEORI

1. Konsep Dasar Sistem Operasi 1.1 Definisi Sistem

Operasi Sistem **Operasi (OS)** adalah perangkat

lunak inti yang berfungsi sebagai perantara antara pengguna, aplikasi, dan perangkat keras komputer. Tanpa sistem operasi, komputer hanyalah kumpulan komponen fisik yang tidak dapat dimanfaatkan secara efektif.

Secara umum, sistem operasi bertanggung jawab untuk mengelola sumber daya sistem seperti prosesor, memori, penyimpanan, dan perangkat input/output. Aplikasi yang dijalankan oleh pengguna tidak berinteraksi langsung dengan perangkat keras; sebaliknya, aplikasi tersebut bergantung pada layanan yang disediakan oleh sistem operasi.

Dalam pendekatan Linux, sistem operasi dipahami sebagai kombinasi dari **kernel** dan serangkaian **utilitas pengguna**.

- Kernel bertanggung jawab untuk mengelola interaksi langsung dengan perangkat keras.
- Utilitas pengguna menyediakan antarmuka dan alat untuk pengguna dan aplikasi.

Meskipun pendekatan ini umumnya dikaitkan dengan Linux, konsep memisahkan inti sistem dari komponen pendukung juga ditemukan di sistem operasi lain seperti Windows dan macOS, meskipun dengan arsitektur dan terminologi yang berbeda.

Dari perspektif fungsional, sistem operasi dapat dianalogikan sebagai **pengatur lalu lintas** dalam sistem komputer. Sistem operasi menentukan:

- Proses mana yang diizinkan menggunakan CPU dan kapan proses tersebut dijadwalkan
- Berapa banyak memori yang dapat dialokasikan untuk setiap aplikasi
- Kapan data disimpan ke atau diambil dari penyimpanan sekunder
- Perangkat mana yang aktif atau menunggu akses.

Dengan mengelola aspek-aspek ini, sistem operasi memastikan bahwa sumber daya sistem dimanfaatkan secara efisien, terkontrol, dan adil di antara aplikasi.

1.2 Fungsi Utama Sistem Operasi A. Manajemen

Proses Manajemen proses

berkaitan dengan pengendalian dan koordinasi program yang sedang dieksekusi. Sistem operasi bertanggung jawab atas:

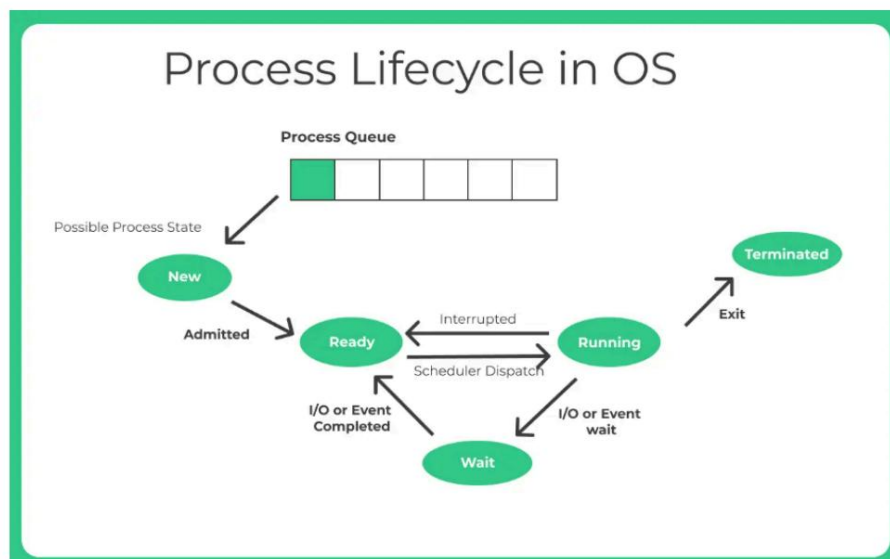
- Membuat dan mengakhiri proses
- Menjadwalkan proses agar

CPU dapat dibagi di antara proses-proses tersebut

- Mengelola status proses (seperti berjalan, menunggu, dan diakhiri)



Tanpa manajemen proses yang tepat, suatu sistem dapat mengalami kondisi seperti **hang** atau **starvation**, di mana satu proses memonopoli CPU untuk jangka waktu yang lama.



Gambar 1. Diagram Siklus Hidup Proses

B. Manajemen Memori

Manajemen memori bertanggung jawab untuk memastikan bahwa setiap aplikasi menerima ruang memori yang cukup tanpa mengganggu proses lain. Sistem operasi mengelola:

- Alokasi dan dealokasi memori
- Proteksi memori antar proses
- Penggunaan memori virtual ketika memori fisik terbatas

Konsep ini sangat penting untuk menjaga stabilitas sistem, terutama ketika banyak aplikasi berjalan secara bersamaan.

C. Manajemen Penyimpanan dan Sistem Berkas

Sistem operasi mengelola bagaimana data disimpan dan diorganisasikan pada media penyimpanan. Fungsi ini meliputi:

- Mengatur struktur file dan direktori
- Mengelola ruang penyimpanan yang tersedia dan terpakai
- Mengontrol izin akses ke file

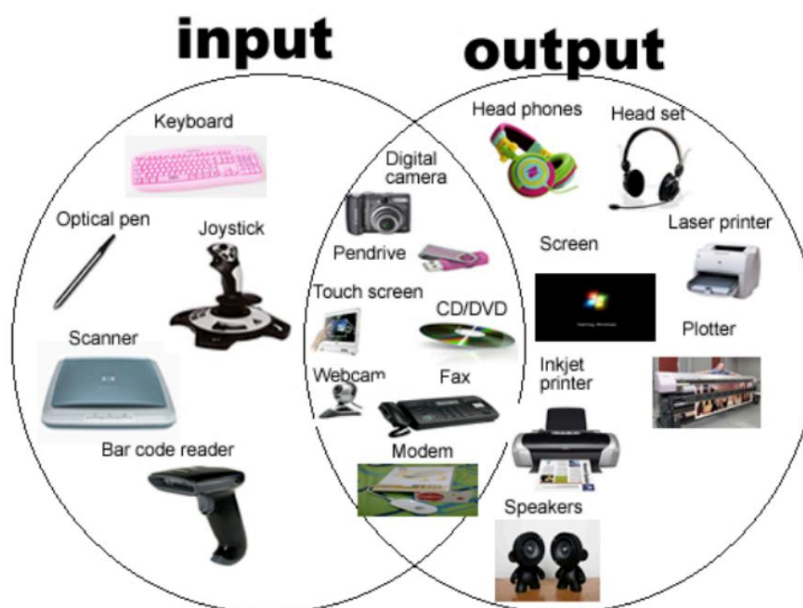
Meskipun sistem operasi yang berbeda menggunakan sistem file yang berbeda, semuanya memiliki tujuan yang sama: untuk menyimpan data dengan cara yang terstruktur, aman, dan andal.

D. Manajemen Perangkat dan Input/Output



Perangkat input/output seperti keyboard, mouse, printer, dan disk tidak diakses langsung oleh aplikasi. Sebaliknya, sistem operasi menyediakan **driver perangkat** dan **mekanisme I/O** yang memungkinkan perangkat keras digunakan tanpa mengharuskan aplikasi untuk memahami detail spesifik perangkat keras.

Melalui abstraksi ini, aplikasi dapat berjalan di berbagai platform perangkat keras tanpa modifikasi yang signifikan.



Gambar 2. Perangkat Input/ Output

E. Keamanan dan Kontrol Akses

Sistem operasi juga bertanggung jawab untuk melindungi sistem dari akses yang tidak sah. Hal ini dicapai melalui:

- Mekanisme otentikasi pengguna
- Manajemen izin file dan sistem
- Isolasi proses untuk mencegah interferensi antar aplikasi

Keamanan sistem operasi sangat penting, terutama di lingkungan multi-pengguna dan sistem yang terhubung ke jaringan.

1.3 Karakteristik Linux sebagai Sistem Operasi

Linux adalah salah satu sistem operasi yang paling banyak digunakan, khususnya di lingkungan server dan konteks pengembangan perangkat lunak. Beberapa karakteristik utamanya meliputi:



a. Multi-pengguna

Linux mendukung banyak pengguna yang beroperasi pada sistem secara bersamaan. Setiap pengguna memiliki hak akses dan lingkungan kerja masing-masing, sehingga Linux cocok dan aman untuk sistem yang digunakan bersama.

b. Multitasking Linux

mampu menjalankan beberapa proses secara bersamaan. Sistem operasi mengelola alokasi waktu CPU secara efisien, memungkinkan pengguna untuk menjalankan beberapa aplikasi tanpa memengaruhi kinerja sistem secara keseluruhan secara signifikan.

c. Stabilitas dan Keamanan

Linux terkenal karena tingkat stabilitasnya yang tinggi. Kerusakan sistem relatif jarang terjadi karena isolasi proses yang kuat, manajemen memori yang ketat, dan model keamanan berbasis izin. Karakteristik ini merupakan alasan utama mengapa Linux umum digunakan di server dan sistem yang sangat penting.

d. Arsitektur Modular (Kernel dan Utilitas Pengguna)

Linux menggunakan arsitektur modular di mana kernel dapat diperluas melalui modul yang dapat dimuat, sementara utilitas pengguna dapat dikembangkan secara independen. Pendekatan ini membuat sistem fleksibel, mudah dipelihara dan diperluas, serta efisien dan mudah beradaptasi dengan kebutuhan pengguna.

2. Struktur Sistem Komputer Struktur sistem

komputer menjelaskan bagaimana **komponen perangkat keras dan perangkat lunak berinteraksi** untuk melakukan tugas komputasi. Memahami struktur ini sangat penting untuk melihat peran **Sistem Operasi (OS)** secara komprehensif, bukan hanya sebagai perangkat lunak, tetapi sebagai koordinator sistem secara keseluruhan.

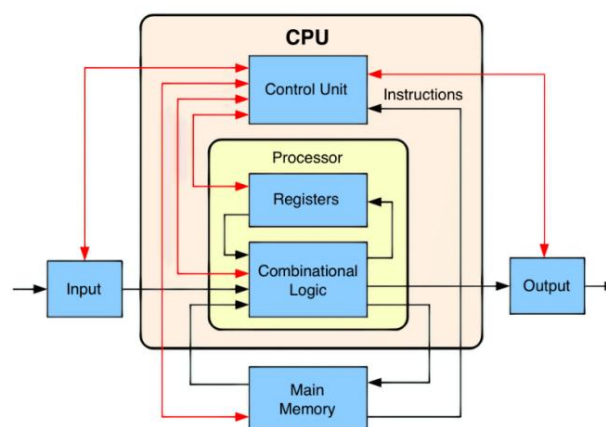
2.1 Komponen Perangkat Keras a.

Unit Pemroses Sentral (CPU)

Unit Pemroses Sentral (CPU) adalah inti dari sistem komputer yang bertanggung jawab untuk mengeksekusi instruksi program. CPU melakukan operasi mendasar seperti perhitungan aritmatika, operasi logika, dan pengendalian alur eksekusi program.

Meskipun CPU beroperasi dengan kecepatan sangat tinggi, ia tidak dapat berfungsi secara independen. Namun, ia membutuhkan sistem operasi untuk menentukan proses mana yang akan dieksekusi, mengelola peralihan konteks antar program, dan mengoptimalkan pemanfaatan waktu pemrosesan.





Gambar 3. Diagram Komponen CPU

b. Memori Utama (RAM)

Memori utama, yang biasa dikenal sebagai RAM, berfungsi sebagai penyimpanan sementara untuk program yang sedang berjalan dan data yang sedang diproses. RAM bersifat **volatil**, artinya isinya akan hilang saat komputer dimatikan. Karena kapasitasnya yang terbatas, manajemen memori yang efektif diperlukan agar beberapa aplikasi dapat berjalan secara bersamaan tanpa konflik.

c. Penyimpanan (Disk)

Penyimpanan digunakan untuk **menyimpan data jangka panjang**, termasuk sistem operasi, aplikasi, dan file pengguna. Media penyimpanan umum meliputi hard disk drive (HDD) dan solid-state drive (SSD). Tidak seperti RAM, penyimpanan bersifat **non-volatil**, sehingga data tetap tersedia bahkan saat sistem dimatikan. Sistem operasi memainkan peran penting dalam mengatur, mengambil, dan melindungi data yang tersimpan pada perangkat ini.

d. Perangkat Input dan Output (I/O)

Perangkat input dan output memungkinkan interaksi antara pengguna dan sistem komputer. Perangkat input seperti keyboard, mouse, dan scanner memungkinkan pengguna untuk memberikan data, sementara perangkat output seperti monitor, printer, dan speaker menyajikan informasi yang telah diproses. Karena setiap perangkat memiliki karakteristik yang berbeda, sistem operasi bertanggung jawab untuk mengelolanya secara seragam melalui mekanisme I/O khusus.

2.2 Peran Sistem Operasi dalam Manajemen Perangkat Keras

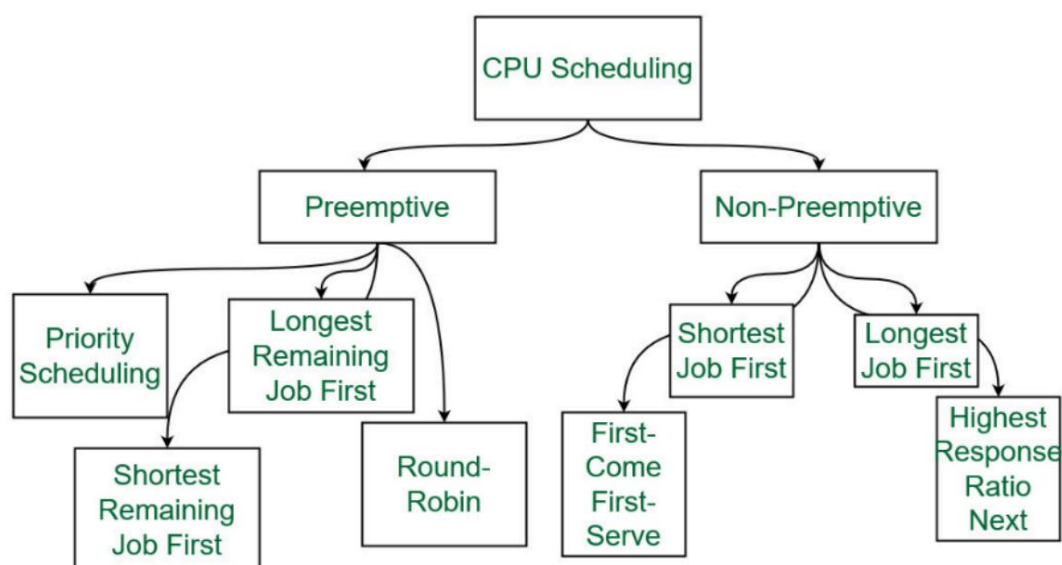
Sistem operasi berfungsi sebagai **pengendali utama interaksi perangkat keras**, memungkinkan aplikasi untuk beroperasi tanpa akses langsung ke komponen perangkat keras fisik.

a. Penjadwalan CPU

Karena sebuah CPU hanya dapat mengeksekusi satu instruksi pada satu waktu, sistem operasi menggunakan **teknik penjadwalan** untuk mendistribusikan waktu CPU di antara beberapa proses. Penjadwalan memastikan



agar setiap proses mendapatkan waktu eksekusi, sistem tetap responsif, dan tidak ada satu proses pun yang memonopoli CPU dalam durasi yang berlebihan.



Gambar 4. Diagram Penjadwalan Proses

b. Isolasi dan Perlindungan Memori

Sistem operasi memastikan bahwa setiap proses hanya mengakses ruang memori yang dialokasikan untuknya sendiri dan tidak dapat membaca atau memodifikasi memori proses lain. Isolasi memori ini sangat penting untuk mencegah kesalahan dalam satu aplikasi mengganggu seluruh sistem.

c. Akses Disk melalui Sistem Berkas

Aplikasi tidak berinteraksi langsung dengan perangkat penyimpanan. Sebaliknya, sistem operasi menyediakan **sistem berkas** yang bertindak sebagai perantara untuk membaca dan menulis data, mengatur struktur direktori, dan mengontrol izin akses berkas. Melalui abstraksi ini, detail fisik perangkat penyimpanan disembunyikan dari pengguna dan aplikasi.



```
praktikumso@localhost:~$ tree -L 1 /
/
├── afs
├── bin -> usr/bin
├── boot
├── dev
├── etc
├── home
├── lib -> usr/lib
├── lib64 -> usr/lib64
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin -> usr/sbin
├── srv
├── sys
├── tmp
├── usr
└── var

21 directories, 0 files
```

Gambar 5. Struktur Sistem Berkas

d. Manajemen Perangkat melalui Driver dan Interupsi Sistem

operasi berkomunikasi dengan komponen perangkat keras menggunakan **driver perangkat**, yang berfungsi sebagai penerjemah antara OS dan perangkat fisik. Selain itu, perangkat keras menghasilkan **interupsi** untuk memberi tahu CPU ketika peristiwa tertentu terjadi, seperti input keyboard atau penyelesaian operasi I/O. Mekanisme ini meningkatkan efisiensi dengan mencegah CPU terus-menerus memeriksa status perangkat.

2.3 Lapisan Sistem

Sistem komputer dapat dilihat sebagai **struktur berlapis**, di mana setiap lapisan memiliki peran dan tanggung jawab spesifik.

a. Pengguna

Pengguna mewakili entitas yang berinteraksi dengan sistem, baik secara langsung maupun melalui aplikasi. Pengguna tidak berkomunikasi langsung dengan perangkat keras, melainkan melalui lapisan perangkat lunak perantara.

b. Aplikasi

Aplikasi adalah program perangkat lunak yang dirancang untuk memenuhi kebutuhan pengguna, seperti pengolah kata, peramban web, atau alat pengolahan data. Aplikasi ini bergantung pada layanan yang disediakan oleh sistem operasi agar dapat berfungsi dengan benar.

c. Shell

Shell bertindak sebagai **antarmuka antara pengguna dan sistem operasi**, menerima perintah dari pengguna dan meneruskannya ke sistem untuk dieksekusi. Shell dapat diimplementasikan sebagai antarmuka berbasis teks atau antarmuka grafis.



d. Kernel

Kernel adalah **komponen inti dari sistem operasi** yang bertanggung jawab untuk mengelola CPU, memori, dan perangkat keras. Kernel beroperasi pada tingkat perangkat lunak terendah dan berjalan terus menerus selama sistem aktif.

e. Perangkat

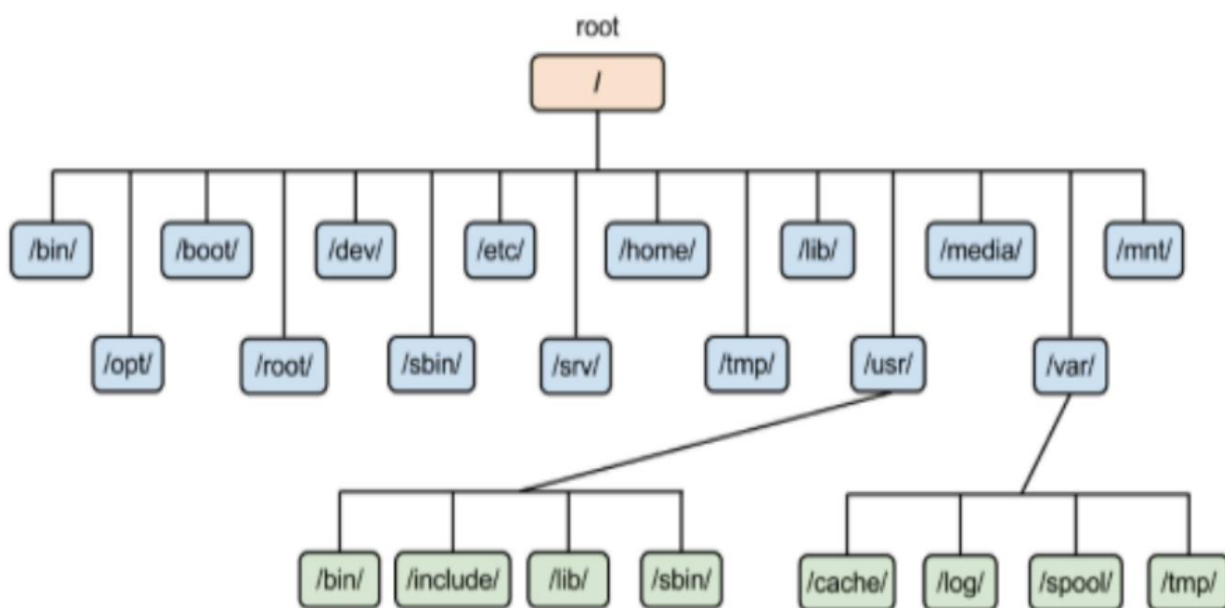
Keras Perangkat keras membentuk lapisan terendah dari sistem dan terdiri dari semua komponen fisik komputer. Semua perintah dan proses pada akhirnya dieksekusi oleh perangkat keras di bawah kendali kernel.

3. Sistem Berkas dan Shell Linux 3.1

Konsep Sistem Berkas Linux

Di Linux, semua data disimpan sebagai **berkas**, termasuk dokumen, gambar, program, dan berkas khusus seperti perangkat atau soket. Berkas diatur ke dalam **direktori (folder)** yang dapat berisi berkas atau subdirektori, membentuk **struktur hierarkis seperti pohon**.

Direktori **root (/)** adalah titik awal dari seluruh sistem file, sedangkan **direktori home (~)** berfungsi sebagai penyimpanan pribadi untuk setiap pengguna. Pemisahan ini memastikan bahwa file sistem berbeda dari data pengguna, sehingga sistem tetap aman dan terorganisir.



Gambar 6. Diagram Hierarki Sistem Berkas Sederhana

3.2 Hierarki Sistem File Linux Dasar Beberapa

direktori penting di Linux meliputi /, /home, /root, /etc, /var, /bin, dan /usr, di mana **direktori root (/)** adalah direktori tingkat atas.



Direktori /	Nama Lengkap	Keterangan
<code>/bin</code>	Biner	Berisi file eksekusi penting (perintah). Tersedia untuk semua pengguna. File-file ini ditandai dengan tanda "x". izin (eksekusi).
<code>/boot</code>	Sistem Berkas Boot	Berisi file-file yang dibutuhkan untuk memboot sistem, seperti... Kernel Linux (<code>vmlinuz</code>), bootloader GRUB, dan boot konfigurasi.
<code>/dev</code>	Sistem berkas perangkat	bertindak sebagai antarmuka ke perangkat keras. Setiap perangkat keras Yang terhubung ke sistem di sini diwakili sebagai sebuah file.
<code>/dli</code>	Dan sebagainya	Lokasi pusat untuk file konfigurasi seluruh sistem. (misalnya, SSH, Cron, jaringan, dan akun pengguna).
<code>/rumah</code>	Direktori Utama	Menyimpan data pribadi dan konfigurasi untuk keperluan rutin. pengguna. Setiap pengguna memiliki subdirektori masing-masing di sini.
<code>/lib</code>	Berkas Perpustakaan	Berisi file pustaka bersama (seperti <code>libc.so</code>) yang dibutuhkan oleh agar file biner di <code>/bin</code> dan <code>/sbin</code> berfungsi.
<code>/media</code>	Media yang Dapat Dilepas	Titik pemasangan default untuk perangkat yang dapat dilepas seperti USB drive, kartu SD, dan CD-ROM.
<code>/mnt</code>	Direktori Pemasangan	Digunakan oleh administrator untuk memasang sementara secara manual. sistem file, seperti hard drive tambahan atau berbagi jaringan.
<code>/memilih</code>	Perangkat Lunak Opsional	Dicadangkan untuk paket perangkat lunak "tambahan" atau pihak ketiga yang tidak mengikuti tata letak file standar.
<code>/proc</code>	Informasi Proses	Sistem file virtual yang menyediakan data waktu nyata tentang proses yang sedang berjalan dan status kernel (misalnya, <code>/proc/cpuinfo</code>).
<code>/akar</code>	Beranda Utama	Direktori home pribadi khusus untuk pengguna root . (Superuser). Ini terpisah dari <code>/home</code> .
<code>/berlari</code>	Direktori Runtime	Menyimpan data runtime yang mudah berubah untuk aplikasi yang sedang berjalan. sejak booting terakhir (misalnya, ID proses dan file kunci).
<code>/sbin</code>	Sistem Biner	Berisi biner sistem penting yang digunakan untuk Administrasi dan pemeliharaan. Biasanya memerlukan akses root. hak istimewa.
<code>/srv</code>	Data Layanan	Menyimpan data untuk layanan yang disediakan oleh sistem, seperti berkas server web (<code>/srv/www</code>) atau data FTP.
<code>/tmp</code>	File Sementara: Tempat	untuk file sementara. File di sini biasanya akan dihapus. secara otomatis saat sistem dihidupkan ulang.
<code>/pengguna</code>	Sistem Unix Sumber daya	Berisi aplikasi, pustaka, dan yang diinstal oleh pengguna. dokumentasi. Ini adalah direktori terbesar di sebagian besar sistem.
<code>/var</code>	Data Variabel	Berfungsi untuk menyimpan file yang ukurannya sering berubah, seperti file log. (<code>/var/log</code>), antrean email, dan cache aplikasi.



Hierarki ini menyederhanakan **pengorganisasian file, manajemen izin, dan pemisahan data sistem dan pengguna.**

3.3 Shell dan Antarmuka Baris Perintah Shell

bertindak sebagai **antarmuka antara pengguna dan sistem operasi**. Melalui shell, pengguna dapat menjalankan program, mengelola file dan direktori, serta mengakses utilitas sistem. Sistem Linux modern biasanya menggunakan **Bash** sebagai shell default, yang mendukung perintah dasar, scripting untuk otomatisasi, serta variabel dan kontrol aliran.

Direktori **kerja** mengacu pada direktori saat ini tempat pengguna beroperasi. Misalnya:

```
praktikumso@localhost:~$ pwd
/home/praktikumso
praktikumso@localhost:~$
```

Perintah-perintah ini masing-masing menampilkan direktori saat ini dan mengubah direktori kerja.

Pengguna dapat menjalankan perintah relatif terhadap direktori kerja atau menggunakan jalur absolut.

4. Navigasi Sistem File Dasar (CLI)

4.1 Konsep Path Di

Linux, lokasi suatu file atau direktori ditentukan menggunakan **path**, yang bisa berupa **path absolut** atau **relatif**.

Jalur absolut dimulai dari akar / dan selalu menunjuk ke lokasi yang sama.

```
praktikumso@localhost:~$ pwd
/home/praktikumso
```

Jalur relatif menentukan lokasi file relatif terhadap direktori kerja saat ini.

```
praktikumso@localhost:~$ ls
Desktop  Downloads  Pictures  Templates
Documents Music      Public    Videos
praktikumso@localhost:~$ cd Documents
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$
```

Jika direktori kerja saat ini adalah `/home/praktikumso`, perintah `cd Documents` akan mengarahkan Anda ke `/home/praktikumso/Documents`.

Simbol khusus yang sering digunakan untuk

navigasi: `..` y direktori induk. Misalnya, naik satu tingkat.



```
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$ cd ..
praktikumso@localhost:~$ pwd
/home/praktikumso
```

~ ÿ direktori beranda pengguna. Misalnya, pindah ke direktori beranda Anda:

```
praktikumso@localhost:~/Documents$ pwd
/home/praktikumso/Documents
praktikumso@localhost:~/Documents$ cd ~
praktikumso@localhost:~$ pwd
/home/praktikumso
```

4.2 Perintah Navigasi Dasar

Memerintah	Fungsi
<code>pwd</code>	Menampilkan direktori kerja saat ini
<code>ls</code>	Daftar file dan direktori
<code>ls -l</code>	Menampilkan daftar file dan direktori beserta detailnya (izin akses, ukuran, tanggal modifikasi).
<code>ls -a</code>	Daftar semua file (termasuk file tersembunyi)
<code>CD</code>	Direktori perubahan
<code>mkdir</code>	Membuat direktori baru
<code>rmdir</code>	Menghapus direktori kosong
<code>rm</code>	Menghapus sebuah file
<code>rm -r</code>	Menghapus direktori beserta isinya.
<code>menyentuh</code>	Membuat berkas kosong baru
<code>cp</code>	Menyalin file atau direktori
<code>cp -r</code>	Menyalin direktori beserta isinya.
<code>mv</code>	Memindahkan atau mengganti nama file/direktori
<code>kucing</code>	Menampilkan isi sebuah file
<code>kepala</code>	Menampilkan beberapa baris pertama dari sebuah file.
<code>ekor</code>	Menampilkan beberapa baris terakhir dari sebuah file.
<code>pohon</code>	Menampilkan struktur direktori sebagai pohon.
<code>jernih</code>	Membersihkan layar terminal



Dengan tabel ini, kami berharap Anda dapat berlatih **menavigasi sistem file, membuat/menghapus file & folder, melihat isi file, menyalin/memindahkan file, dan mengelola izin**, yang merupakan keterampilan penting untuk bekerja di CLI Linux.

LATIHAN 1: Sang Pelancong (Konsep Jalan)

1. Periksa lokasi Anda saat ini.
2. Buat direktori bernama **Exercise**.
3. Masuk ke direktori **Latihan** .
4. Naik satu tingkat direktori.
5. Masuk kembali ke direktori **Latihan** .
6. Tetap berada di direktori saat ini.
7. Naik dua tingkat direktori.
8. Lompat ke direktori sistem menggunakan jalur absolut.
9. Kembali ke direktori utama Anda.

LATIHAN 2: Sang Detektif (Menjelajahi Hasil)

Mulailah dengan membuka terminal Anda dan tanyakan kepada sistem di mana Anda berada dengan mengetikkan ``pwd``. Untuk melihat apa yang ada di sekitar Anda, ketik ``ls`` untuk mendapatkan daftar nama sederhana. Sekarang, mintalah "cerita lengkap" dengan mengetikkan ``ls -l``. Perhatikan bagaimana **shell** menampilkan detail tambahan seperti izin dan ukuran file yang sebelumnya tersembunyi.

LATIHAN 3: Sang Arsitek (Manajemen File)

1. Buat file kosong baru bernama `draft.txt` menggunakan perintah `touch` .
 2. Buat file duplikat bernama `backup.txt`.
 3. Ubah nama file menjadi `final.txt` . 4.
- Terakhir, periksa pekerjaan Anda untuk melihat apakah ada konten di dalamnya.

LATIHAN 4: Pembersihan (Pemeliharaan Sistem)

1. Hapus file individual bernama `final.txt`.
2. Gunakan perintah dengan flag rekursif untuk menghapus seluruh direktori `Latihan` dan semua isinya.
3. Verifikasi bahwa direktori tersebut telah hilang dengan mencoba mencantumkan isi direktori Anda saat ini. lokasi.

LATIHAN 5: Pengamat Akar (Folder Bersarang & Pohon Sistem)

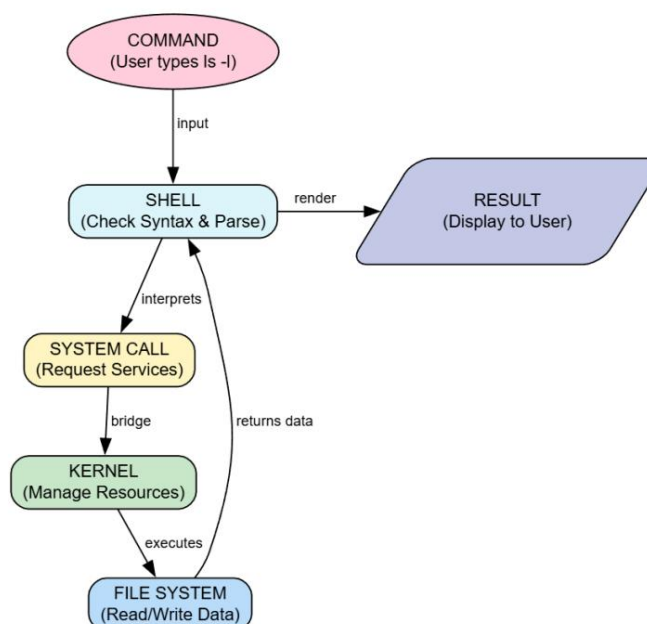
1. Membuat folder di dalam folder lain dalam satu perintah
2. Masuk ke folder terdalam dan verifikasi jalur Anda dengan perintah ``pwd``.
3. Sekarang, mari kita lihat gambaran besar sistem Linux dengan menampilkan struktur tingkat pertama direktori root.



4.3 Alur Kerja Perintah

Setiap perintah CLI melewati beberapa tahapan sebelum menghasilkan tindakan pada sistem:

1. **Perintah:** pengguna mengetikkan perintah di terminal.
2. **Shell:** shell menerima perintah dan memeriksa sintaksnya.
3. **Panggilan Sistem:** shell meminta layanan sistem dari kernel sesuai dengan memerintah.
4. **Kernel:** kernel mengeksekusi instruksi dan mengelola sumber daya perangkat keras.
5. **Sistem Berkas:** sistem berkas melakukan operasi yang diminta, seperti membaca sebuah Perintah tersebut mengambil direktori, membuat folder, atau menampilkan isi file, kemudian mengembalikan hasilnya ke shell dan menampilkannya kepada pengguna.



Gambar 7. Alur Kerja Perintah



APLIKASI PRAKTIS

LATIHAN 1

Skenario: Anda perlu membuat struktur folder untuk proyek mata kuliah Sistem Operasi yang memisahkan laporan dari skrip latihan.

- **Langkah 1:** Buat folder utama bernama **ProjectOS**.
- **Langkah 2:** Di dalam **ProjectOS**, buat dua subdirektori sekaligus bernama **Reports** dan **Scripts**.
- **Langkah 3:** Masuk ke folder **Scripts**, lalu buat file kosong bernama **test.sh**.
- **Langkah 4:** Tanpa keluar dari folder **Scripts**, gunakan **Relative Path** untuk menyalin file **test.sh** ke dalam folder **Scripts**.
Folder **Laporan** dengan nama baru, **backup_test.txt**.
- **Langkah 5:** Gunakan pintasan khusus untuk langsung kembali ke **Direktori Utama Anda**.

LATIHAN 2

Skenario: Anda ingin memastikan semua file tertata dengan benar dan menghapus file yang tidak perlu secara efisien.

- **Langkah 1:** Gunakan perintah **Jalur Absolut** untuk berpindah ke folder **ProjectOS/Reports** Anda dibuat sebelumnya.
- **Langkah 2:** Tampilkan isi folder tersebut dengan **detail lengkap** (termasuk ukuran file dan waktu pembuatan) untuk memastikan bahwa **backup_test.txt** ada.
- **Langkah 3:** Ganti nama file **backup_test.txt** menjadi **final_report.txt**.
- **Langkah 4:** Pindah satu tingkat ke **direktori induk** menggunakan simbol khusus.
- **Langkah 5:** **Hapus** seluruh folder **ProjectOS** beserta semua isinya secara permanen.
perintah tunggal.

DEMO - STUDI KASUS

Situasi Seorang

pengguna sedang bekerja di sistem Linux dan ingin membuat struktur proyek sederhana yang memisahkan dokumen, skrip, dan file data. Tujuannya adalah untuk menjaga agar sumber daya proyek tetap terorganisir dan mudah dikelola saat bekerja sepenuhnya dari terminal.

Masalahnya

adalah berkas-berkas proyek belum tersusun dalam struktur direktori yang jelas. Pengguna perlu memahami cara membuat direktori, membuat berkas, menavigasi direktori, menerapkan jalur absolut dan relatif, serta mengidentifikasi direktori kerja saat ini agar dapat membangun tata letak proyek yang dibutuhkan dengan benar.

Misi



Pengguna ingin membuat struktur proyek sederhana dengan folder untuk dokumen, skrip, dan data. Tujuannya adalah untuk mengatur file proyek secara efisien sambil memahami navigasi direktori, jalur absolut dan relatif, serta direktori kerja di Linux.

Struktur Proyek:

Proyek/

 yyy Dokumen/ y yyy

 report1.txt y yyy

 report2.txt yyy Skrip/

 y yyy main.sh

 y yyy helper.sh

 yyy Data/ yyy

 data1.csv

 yyy data2.csv

 yyy data3.csv

Aturan Demo

1. **Dilarang Mengakses Modul atau Catatan** Mahasiswa dilarang membuka PDF modul, catatan pribadi, atau dokumentasi online selama sesi demo.
2. **Mendemonstrasikan Keterampilan Navigasi Linux** Inti Siswa harus secara jelas menunjukkan kemahiran dalam:
 - Menavigasi sistem file
 - Perpindahan direktori yang efisien
 - Mengelola file dan direktori, termasuk membuat (**mkdir**, **touch**), menyalin (**cp**), memindahkan (**mv**), dan menghapus (**rm**).
 - Menjelajahi struktur sistem dan melihat isi file
3. **Verifikasi Akhir** Setelah menyelesaikan tugas yang diberikan, siswa harus memverifikasi hasilnya (misalnya, mengkonfirmasi bahwa file telah dipindahkan atau direktori telah dihapus) untuk memastikan keadaan sistem sesuai dengan persyaratan misi.
4. **Penilaian Konseptual Langsung** Selama atau setelah demonstrasi, para asisten akan mengajukan beberapa pertanyaan.

Pertanyaan-pertanyaan kunci untuk mengevaluasi **pemahaman konseptual Anda**.

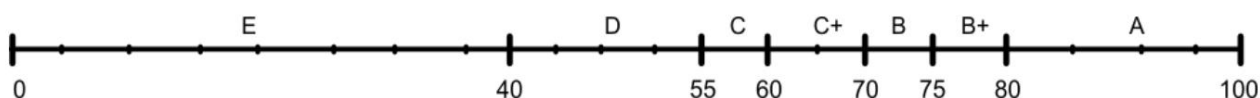
Catatan: Harap persiapkan diri dengan matang, karena ini merupakan bagian penting dari nilai Anda.
5. **Ikuti Instruksi Langsung** Semua tindakan harus dilakukan secara interaktif mengikuti arahan instruktur atau asisten.



RUBRIK PENILAIAN

Aspek Penilaian	Poin (Total 100%)
LATIHAN	Total 5%
Latihan 1	1%
Latihan 2	1%
Latihan 3	1%
Latihan 4	1%
Latihan 5	1%
PRAKTIK	Total 20%
Latihan 1 (Penilaian verbal)	5%
Latihan 2 (Penilaian verbal)	5%
Penyelesaian Latihan (Verifikasi Langsung)	10%
DEMO	Total 75%
Penyelesaian Studi Kasus	35%
Memahami Logika Manipulasi Direktori	15%
Memahami Penguasaan Logika Navigasi	10%
Validasi Keluaran	10%
Penyelesaian Masalah	5%

SKALA PENILAIAN



A = (81 - 100) ÿ Sepuh

B+ = (75 - 80) ÿ Sangat Baik

B = (70 - 74) ÿ Baik

C+ = (60 - 69) ÿ Cukup Baik

C = (55 - 59) ÿ Adil

D = (41 - 54) ÿ Miskin

E = (0 - 40) ÿ Bro sungguh...

